

Phase 1: Hardware & Media Access

Presentation Confidence Detector

Project Blueprint

Phase 1: Hardware & Media Access

What Is This Phase?

Before you can detect confidence, nervousness, or anything else, you need **raw material** — live video from the camera and live audio from the microphone. This phase teaches you how to access hardware for the **browser-first MVP**, how to handle permissions gracefully, and how to clean up resources properly. The same concepts can later transfer to desktop or mobile builds.

In ExamGuard, you used OpenCV's `cv2.VideoCapture()` to read camera feeds. The Confidence Detector runs in the **browser**, so everything changes — you use the **MediaDevices API** instead. Same concept (get frames from camera), completely different tools.

WHY This Phase Matters for the Confidence Detector

```
User clicks "Start Practice Session"
|
v
[Browser requests camera + mic permission] ← getUserMedia
|
v
[Video stream → face detection pipeline] ← MediaStream video track
[Audio stream → speech + voice analysis] ← MediaStream audio track
|
v
[Canvas overlay draws landmarks on face] ← Canvas API
|
v
[User clicks "Stop"]
|
v
[Camera + mic released properly] ← Stream cleanup
```

Without this phase, you have no video to analyze faces from, no audio to analyze voice from, and no way to draw feedback overlays. **This is the foundation everything else sits on.**

Skills to Learn

1. `getUserMedia` API — Accessing Camera and Microphone

What is it?	<code>navigator.mediaDevices.getUserMedia()</code> is the browser API that requests access to the user's camera and/or microphone. It returns a <code>MediaStream</code> — a live feed of video and audio data.
WHY it matters	The Confidence Detector needs BOTH camera (for face/expression detection) and microphone (for speech/voice analysis) running simultaneously. This single API call gets you both.
Difficulty	Easy — the API itself is simple. The complexity is in handling edge cases (denied permission, no

	camera, multiple cameras).
--	----------------------------

What to learn:

You call `navigator.mediaDevices.getUserMedia()` with a configuration object specifying video and audio options (resolution, facing mode, echo cancellation). The returned stream is then attached to a `<video>` element's `srcObject` property so the live feed displays on the page. You can pass `facingMode: 'user'` to select the front camera on mobile devices.

Confidence Detector connection: This is the very first line of code that runs when a user starts a practice session. You request video (for face analysis) and audio (for speech analysis) in one call. The `facingMode: 'user'` constraint ensures you get the front camera on phones — the one pointing at the speaker's face.

2. MediaStream and Tracks — Video Track vs Audio Track

What is it?	A <code>MediaStream</code> contains one or more <code>MediaStreamTrack</code> objects. Each track is either a video track or an audio track. You can access, enable/disable, or stop them independently.
WHY it matters	The Confidence Detector sends the video track to the face/expression pipeline and the audio track to the speech/voice pipeline. They are processed separately but run in parallel. You need to understand tracks to route data correctly.
Difficulty	Easy — but critical to understand. Many beginners treat the stream as one blob and never learn to work with individual tracks.

What to learn:

After getting a `MediaStream`, you extract individual video and audio tracks using `getVideoTracks()` and `getAudioTracks()`. Each track has properties like `label` (device name), `readyState` (live or ended), and an `enabled` flag you can toggle to mute/unmute without stopping the stream. You can also call `getSettings()` on a track to see the actual resolution and frame rate being delivered.

Confidence Detector connection: You will pass `videoTrack` to `MediaPipe FaceMesh` for expression detection and route `audioTrack` to the Web Audio API for pitch/volume analysis AND to the Speech Recognition API for transcription. Understanding tracks means you can control each independently — for example, letting the user mute their mic without stopping the camera.

3. Permission Handling — What Happens When the User Denies

What is it?	When you call <code>getUserMedia()</code> , the browser shows a permission popup. The user can
--------------------	--

	Allow, Block, or dismiss it. Your app must handle ALL of these cases gracefully.
WHY it matters	If a user denies camera access and your app crashes or shows a blank screen, that is a terrible experience. The Confidence Detector should explain what is needed and why, and degrade gracefully (e.g., audio-only mode if camera is denied).
Difficulty	Medium — the logic is simple, but there are platform-specific quirks (Safari vs Chrome vs Firefox, HTTP vs HTTPS, iframe restrictions).

What to learn:

You wrap the `getUserMedia()` call in a try/catch and check the error's `name` property to determine what went wrong. The main error types are `NotAllowedError` (user denied permission), `NotFoundError` (no camera/mic detected), `NotReadableError` (device in use by another app), and `OverconstrainedError` (requested settings not supported, so you fall back to defaults). Each case shows a user-friendly message explaining the problem and what to do.

Confidence Detector connection: A real product handles errors. Your app needs to tell the user WHY it needs their camera ("to analyze facial expressions and eye contact") and provide fallback modes. This is the difference between a demo and a usable tool.

4. Stream Cleanup — Stopping Camera and Mic Properly

What is it?	When the user ends a session, you must explicitly stop every track in the stream. If you do not, the camera light stays on, the mic keeps recording, and the browser tab shows a recording indicator.
WHY it matters	In ExamGuard, there was a bug where the camera was not released properly, causing issues on subsequent sessions. You already learned this lesson the hard way. The Confidence Detector must not repeat this mistake.
Difficulty	Easy to do, easy to forget. The pattern is simple, but you must call it in EVERY exit path (user clicks stop, navigates away, closes tab, error occurs).

What to learn:

You create a `stopStream` function that iterates over all tracks in the stream and calls `track.stop()` on each one. This function must be called in every exit path: when the user clicks stop, when they navigate away (via the `beforeunload` event), and when a fatal error occurs. In React, you start the

camera inside a `useEffect` and return a cleanup function that stops all tracks when the component unmounts, ensuring the camera and mic are always properly released.

Confidence Detector connection: The React `useEffect` cleanup pattern is exactly what you will use. When the user leaves the practice session page or ends their session, every track must be stopped. No camera light left on. No mic left recording. **You learned this from the ExamGuard camera bug — apply that lesson here.**

5. Canvas API — Drawing Overlays on Video

What is it?	The HTML <code><canvas></code> element lets you draw shapes, lines, text, and images programmatically. By layering a canvas on top of the video feed, you can draw face landmarks, bounding boxes, and feedback text directly onto the live view.
WHY it matters	The Confidence Detector needs to show the user WHERE it is looking — draw dots on facial landmarks, circles around eyes for eye contact tracking, and a bounding box around the detected face. This is the browser equivalent of OpenCV's <code>cv2.rectangle()</code> and <code>cv2.putText()</code> that you used in ExamGuard.
Difficulty	Medium — basic shapes are easy, but syncing canvas drawing with the video frame rate and scaling correctly takes practice.

What to learn:

You layer a `<canvas>` element on top of the `<video>` element and get a 2D drawing context. In a `requestAnimationFrame` loop, you clear the canvas each frame, then draw a green bounding box around the detected face, red dots at each landmark point (nose tip, eye corners, etc.), and white text showing feedback like eye contact status and confidence score. This loop runs continuously at the video's frame rate, creating a real-time overlay on the live camera feed.

Confidence Detector connection: This is how you show the user real-time visual feedback. When MediaPipe FaceMesh returns 468 landmark points, you draw them on the canvas overlay. When the system detects poor eye contact, you highlight the eye region in red. The canvas is your "drawing board" layered over the live video.

WHY Browser-First?

Before you start building, understand WHY the Confidence Detector runs in the browser instead of as a desktop app or mobile app. This is a deliberate architectural decision, not a default.

Factor	Browser (our choice)	Desktop (e.g., Python + OpenCV)	Mobile (native app)
--------	----------------------	---------------------------------	---------------------

Installation	Zero. User opens a URL. Done.	User installs Python, pip packages, resolves dependency conflicts. 15-30 min setup.	User downloads from app store. Review process takes days/weeks.
Privacy	All processing stays in the browser. Video never leaves the device.	Same — local processing.	Same — local processing.
Deployment	Push to hosting, everyone gets the update instantly.	User must re-download or update manually.	App store review for every update.
Cross-platform	Works on Windows, Mac, Linux, Chromebook — anything with Chrome.	Must test on each OS. Python version conflicts.	Separate iOS and Android builds.
Hardware access	Camera + mic via <code>getUserMedia</code> . Sufficient for our needs.	Full hardware access. Multiple cameras, custom drivers.	Camera + mic + accelerometer, GPS, etc.
Processing power	Limited by browser sandbox. WebGL helps but not as fast as native.	Full CPU/GPU access. Best for heavy ML models.	Limited by phone hardware. Battery drain.
Offline support	Possible with service workers, but harder.	Works fully offline by default.	Works offline once installed.

Conclusion: For an MVP demo that anyone can try instantly with zero setup, browser wins decisively. The user clicks a link, grants camera/mic permission, and starts practicing. No installs, no dependencies, no app store. When you need heavier processing later (bigger ML models, longer sessions), you can add a server backend while keeping the browser as the front end. Browser-first is the fastest path from "idea" to "someone is using this."

Resolution Tradeoffs

The camera resolution you request in `getUserMedia` directly affects how fast your face detection pipeline runs. Higher resolution means more pixels to process per frame, which means slower frame rates.

Resolution	Pixels per Frame	Typical FPS with MediaPipe	Best For
640 x 480 (VGA)	307,200	30-60 FPS	Real-time analysis (our choice)
1280 x 720 (HD)	921,600	15-30 FPS	Better face detail, but noticeably slower
1920 x 1080 (Full HD)	2,073,600	5-15 FPS	Screenshots/recording, too slow for real-time

We use 640x480. Here is why:

- MediaPipe FaceMesh needs roughly 30+ FPS to feel "real-time" to the user. Below 20 FPS, the overlay visibly lags behind the video and the experience feels broken.
- FaceMesh works fine at 640x480. The 468 landmarks are detected accurately because the model was trained to handle low-resolution faces. You do not gain meaningful landmark accuracy by going to 1280x720.
- Lower resolution also means smaller frames being copied to the canvas, faster drawing, and less memory usage.
- If you later want to RECORD the session for playback, you can request a separate higher-resolution stream for recording while keeping the analysis stream at 640x480.

The math: 1920x1080 has roughly 6.75x more pixels than 640x480. That is 6.75x more work per frame. Going from 30 FPS to 5 FPS is not a tradeoff — it is a dealbreaker for real-time feedback.

HTTPS Requirement

This catches many beginners: **modern browsers REQUIRE a secure context (HTTPS) to access the camera and microphone.** If you deploy your app on plain HTTP, `getUserMedia()` will throw a `NotAllowedError` and you will spend hours debugging permissions when the real problem is your URL.

The exceptions:

URL	Camera/Mic Access?	Why
https://yoursite.com	Yes	Secure context. Production standard.
http://localhost	Yes	Browsers treat localhost as secure for development.
http://127.0.0.1	Yes	Same as localhost — treated as secure.
http://yoursite.com	NO	Not a secure context. <code>getUserMedia</code> blocked.
http://192.168.1.x	NO	Local network IP, but still not secure. Blocked.
file:///path/to/index.html	Depends	Chrome blocks it. Firefox may allow it. Do not rely on this.

For development: Use `localhost` (Vite's dev server runs on `http://localhost:5173` by default — this works fine).

For sharing with others: You need HTTPS. Options include Netlify/Vercel (free HTTPS), ngrok for temporary tunnels, or a self-signed certificate for local network testing. Do not skip this — if your

friend opens your app on their phone via your local IP address over HTTP, the camera will not work and you will both be confused.

Browser Compatibility

Not all browser APIs are supported everywhere. This table shows what works where for the Confidence Detector's needs:

API	Chrome	Edge	Firefox	Safari	Use in Confidence Detector
getUserMedia	Yes	Yes	Yes	Yes (12.1+)	Camera + mic access. Works everywhere modern.
Web Speech API (SpeechRecognition)	Yes	Yes	No	No	Speech-to-text for MVP. Chrome/Edge only.
Web Audio API	Yes	Yes	Yes	Yes	Pitch/volume analysis. Works everywhere.
MediaPipe JS (Face Landmarker)	Yes	Yes	Yes	Yes (with WebGL)	Face detection. Works everywhere modern.
Canvas API	Yes	Yes	Yes	Yes	Drawing overlays. Universal support.
MediaRecorder	Yes	Yes	Yes	Yes (14.1+)	Recording sessions for playback. Broadly supported.

The bottleneck is Web Speech API. It only works reliably in Chrome and Edge. This means:

- For the **MVP**: Target Chrome/Edge users. This covers roughly 75-80% of desktop browsers. Add a browser check on load and show a clear message if the user opens it in Firefox/Safari: "This app requires Chrome or Edge for speech recognition."
- For **v2**: Replace Web Speech API with Vosk (runs via WebAssembly, works in all browsers) or a server-side STT engine. This removes the browser restriction entirely.

- Everything else works everywhere.** Camera access, audio analysis, face detection, and canvas drawing are universally supported. The speech-to-text engine is the only browser-specific dependency.

Skill Summary Table

Skill	What It Does	Confidence Detector Use	Difficulty
<code>getUserMedia()</code>	Access camera + mic	Start practice session	Easy
MediaStream Tracks	Separate video/audio	Route to face vs. speech pipelines	Easy
Permission Handling	Handle denied access	Graceful errors + fallback modes	Medium
Stream Cleanup	Release hardware	Prevent ExamGuard camera bug	Easy
Canvas API	Draw on video overlay	Show landmarks, scores, feedback	Medium

After This Phase

After this phase, you can capture live video and audio in the browser MVP.

You will have a working camera + mic feed in the browser, proper permission handling, clean resource management, and a canvas overlay ready for face landmarks. This is the blank canvas (literally) that Phases 2 and 3 will paint on.

Resources

Official Documentation

Resource	Link	What You Get
MDN: <code>getUserMedia()</code>	https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia	Complete API reference with examples
MDN: MediaStream	https://developer.mozilla.org/en-US/docs/Web/API/MediaStream	Track management, events, properties
MDN: MediaStreamTrack	https://developer.mozilla.org/en-US/docs/Web/API/MediaStreamTrack	Individual track control
MDN: Canvas API	https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API	Drawing shapes, text, images
MDN: Permissions API	https://developer.mozilla.org/en-US/docs/Web/API/Permissions_API	Checking permission state without prompting

Video Tutorials

Resource	Link	What You Get
Traversy Media: Webcam in JS	https://www.youtube.com/watch?v=RHRHRBRp3lQ	Quick practical demo of getUserMedia
The Coding Train: Webcam + Canvas	https://www.youtube.com/watch?v=bkGf4fEHKak	Drawing on video with Canvas
Web Dev Simplified: getUserMedia	https://www.youtube.com/watch?v=_jIGMiWcNYs	Clean explanation of media streams
Fireship: WebRTC in 100 Seconds	https://www.youtube.com/watch?v=WmR9IMUD_CY	Big picture context for media APIs

Practice

Resource	What You Do
Build a selfie app	getUserMedia + Canvas + save frame as image
Build a video recorder	getUserMedia + MediaRecorder API
Build a mirror with filters	getUserMedia + Canvas + CSS filters

Key Takeaway

Hardware access is the **entry point** for every real-time analysis project. In ExamGuard, OpenCV handled this on the Python/desktop side. In the Confidence Detector, the browser's MediaDevices API does the same job. The concepts are identical — request hardware, get a stream, process frames, clean up. The tools are different. Learn these browser APIs well, because every other phase depends on having a working camera and microphone feed.